독감 환자 예측

이진하

요약

본 보고서는 2017년도부터 2025년도까지 주기 별 인플루엔자 신규 입원 환자 수 데이터를 기반으로, 2027년도의 독감 환자 수를 예측하기 위한 시계열 분석을 수행하였다. 계절적 요인이 강한 독감 데이터의 특성을 고려하여 SARIMA 모형을 적용하였으며, 가장 우수한 SARIMA(0,1,0)(1,1,1,12) 모형이 최종 예측 모형으로 선정되었다. 본 연구는 독감 환자 수의 계절성 요인을 반영한 예측 모델을 제시함으로써 향후 독감 환자 증가 시기의 사전 예측, 감염병 대응 및 보건 정책 수립, 보건 인프라 준비 등 도움을 줄 수 있을 것으로 기대된다.

1.서론

1.1 연구배경

인플루엔자(독감)는 매 년 반복적으로 유행하는 계절성 호흡기 감염병이다.
인플루엔자는 발생 시기에 따라 사회적, 경제적 영향을 미칠 수 있기 때문에 감염병의 유행 시점을 아는 것은 매우 중요한 과정이다. 코로나19 이후 해외 유입과 인구 이동이 증가했으며, 거리두기 완화와 코로나19 격리 해제 방침으로 인해 새로운 인플루엔자가 발생하기도 한다. 특히 인플루엔자 A형/B형 독감은 매년 계절적으로 반복되고 있어, 이에 따른 사전 예측과 대비로 감염병을 예방하는 것이 중요하다.
따라서, 본 연구는 2017년도부터 2025년까지의 급성 호흡기 감염증 주기/절기 별 데이터를 활용하여 시계열 분석을 수행하고, 향후 2027년도의 감염자 수와 시기를 예측하고자 한다.
본 분석 결과를 바탕으로 향후 독감 예방 및 대비 정책 수립에 도움이 될 수 있는 시사점을 제시하고자 한다.

1.2 데이터 설명

2017 - 2018 절기 ~ 2024 - 2025 절기까지의 인플루엔자 신규 입원 환자수 데이터

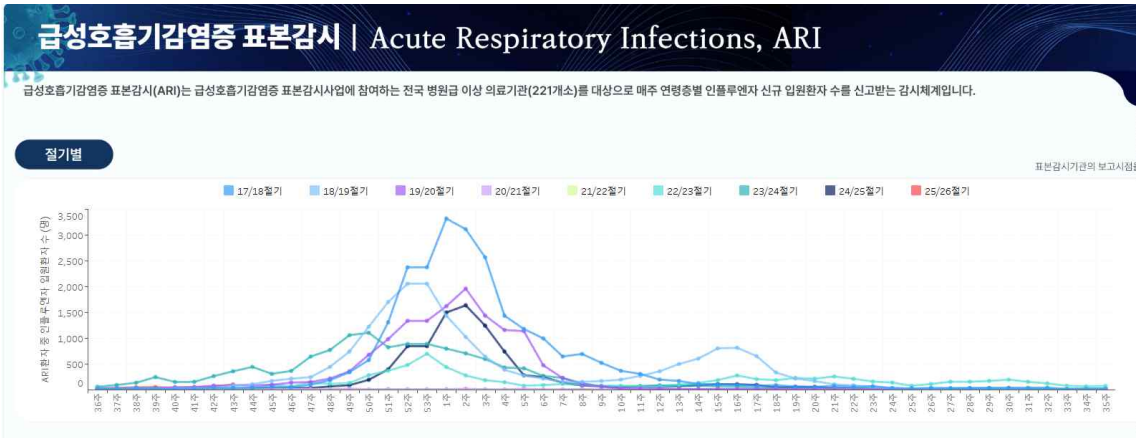


그림 1.1 급성호흡기 감염증 표본 감시 자료

표 1.1 데이터 변환 후 데이터 형태

season	week	value
2017~2018	32	22
2017~2018	33	10
2017~2018	34	13
2017~2018	35	12
2018~2019	36	25
2018~2019	37	14
2018~2019	38	25
2018~2019	39	17
2018~2019	40	19

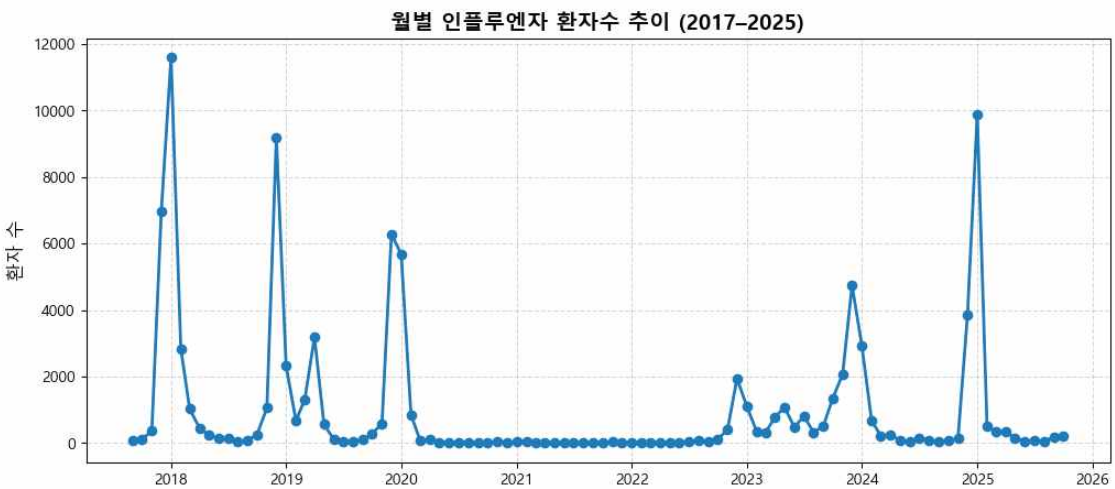
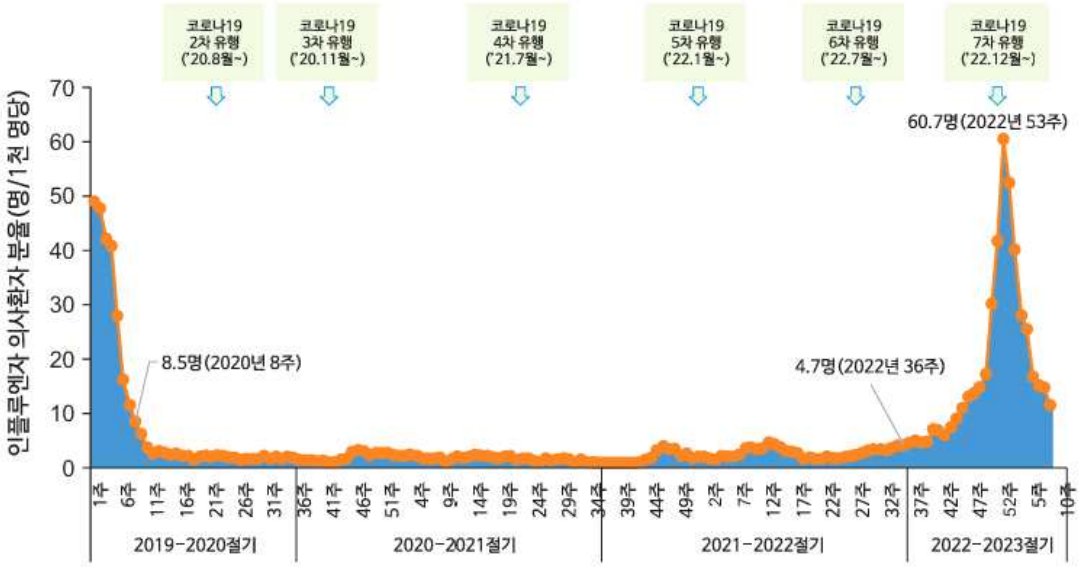


그림 1.2 월 별 인플루엔자 환자수 추이

<그림 1.2> 는 각 절기별 데이터를 하나로 통합하였다. 각 연도 구간마다 파란색 점이 12개인 것을 확인 할 수 있다. 이는 각 월(1월~12월)을 의미한다.
그래프를 통해 인플루엔자 감염자 수가 매년 겨울철(11월~1월)에 증가하는 계절적 유행성을 보이는 것을 확인할 수 있었다.
그러나 2020년 3월부터 2022년 10월까지는 독감 환자 수가 현저히 감소하였다.



출처 : https://djcidc.or.kr/bbsDown/dataBoard_new/2023071971336770332.pdf?utm_source=chatgpt.com

그림 1.3 코로나 유행기의 인플루엔자 환자 분율

그에 따른 근거로, 질병관리청이 발행한 코로나19 대유행 기간(2020년 1주 ~ 2023년 8주)의 인플루엔자 표본 감시 결과를 참고하였다. 해당 보고서에 따르면, 코로나19 유행시기에는 인플루엔자 유행이 발생하지 않은 것으로 확인되었다.
이는 2020년 초부터 코로나19가 유행하기 시작하며 마스크 착용, 사회적 거리두기, 학교 휴교, 백신 의무화 등 방역 조치로 인해 코로나뿐만 아니라 인플루엔자 바이러스 전파도 억제된 것으로 판단된다.
따라서, 본 연구에서는 <그림 1.2>의 2020년 3월부터 2023년 10월까지의 코로나 유행기를 비정상 구간으로 간주하고, 해당 기간의 데이터를 처리하여 분석하기로 하였다.

2. 모형화

2.1 정상성 검정

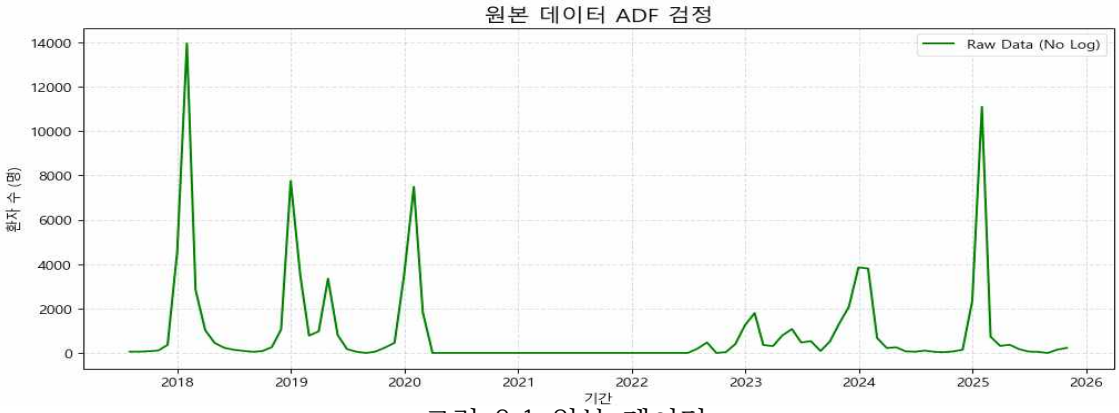


그림 2.1 원본 데이터

<그림 2.1>의 원본 데이터는 p-value의 값이 0.33으로 0.05 이상으로 나왔으며, 데이터가 비정상이므로 분산 안정을 위해 로그변환이 필요하다고 판단하였다.

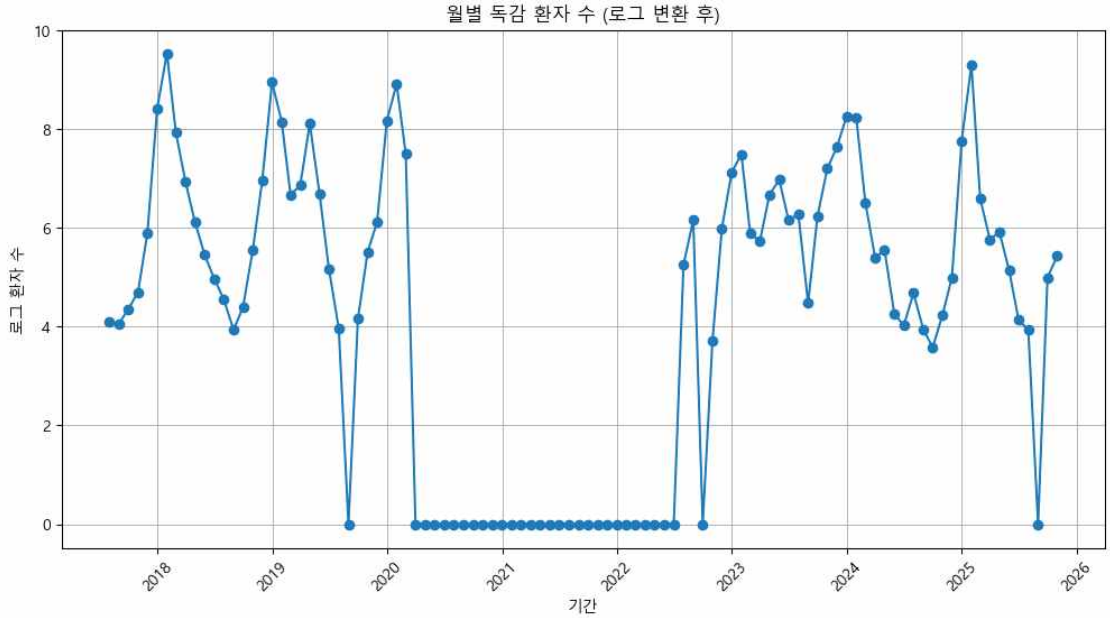


그림 2.2 로그 변환 후 월별 독감 환자 수

<그림 2.2>에서, 로그 변환을 통해 데이터의 분산을 일정하게 맞춘 결과이다. 2020년부터 2022년 사이의 구간이 0으로 유지되는 것을 볼 수 있다. 이는 해당 구간을 삭제하지 않고 0으로 처리해 시계열의 연속성을 유지한 결과이다. 로그 변환 후에도 여전히 계절성이 남아있음을 시각적으로 확인할 수 있었다. 따라서 정상성을 완전히 확보하기 위해 계절차분을 수행하기로 하였다.

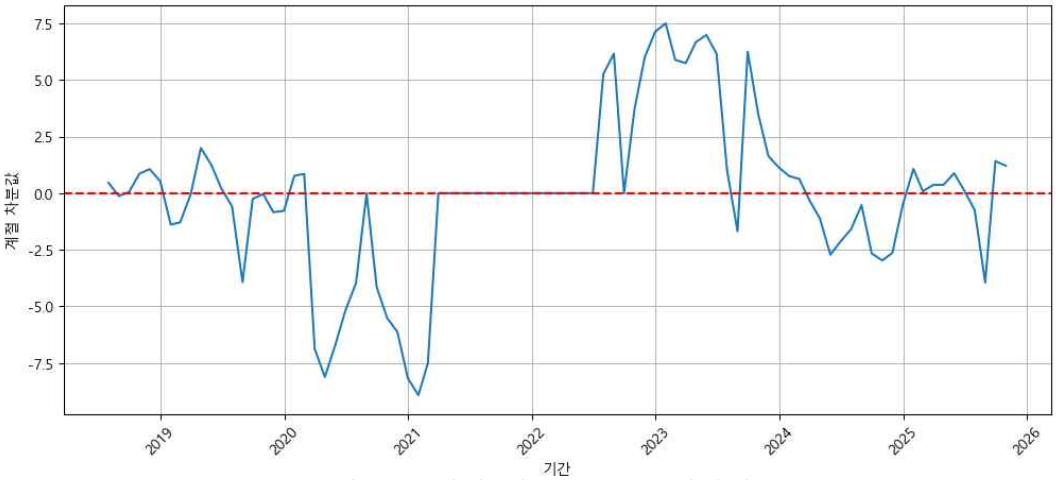


그림 2.3 계절 차분(12) 후 데이터

<그림 2.3>은 독감 데이터의 가장 큰 특징인 12개월 주기의 계절성을 제거하기 위해 계절차분을 먼저 수행한 결과이다. 여전히 뚜렷한 추세가 남아있음을 확인 할 수 있었다. 실제로 ADF검정을 수행한 결과, p-value가 약 0.41로 나타났다. 이는 통계적으로 유의수준 0.05보다 높은 수치로, 아직 데이터가 정상성을 확보하지 못했음을 의미한다. 따라서, 계절 차분만으로는 부족하다고 판단하여 남아있는 추세까지 제거하기 위해 추가적으로 일반 차분도 수행하기로 하였다.

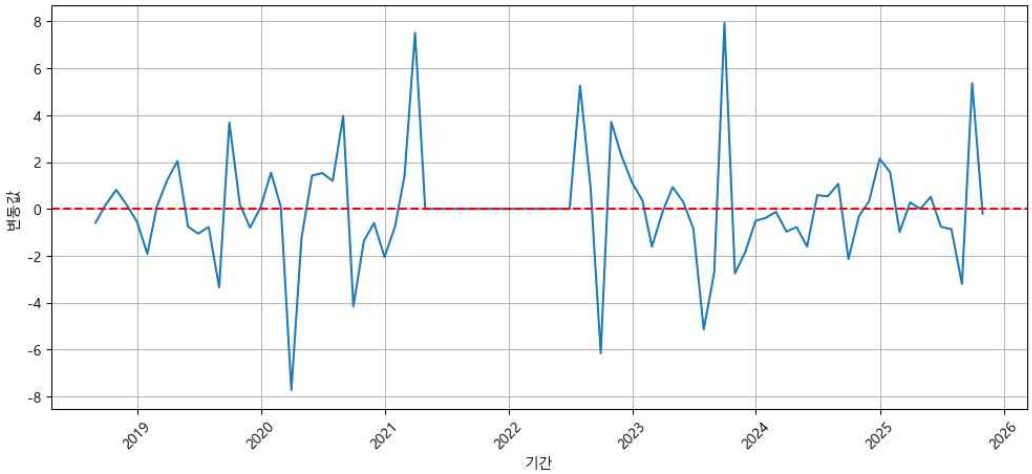


그림 2.4 계절차분(12) + 일반차분(1) 후 데이터

<그림 2.4>에서 비계절 차분과 일반 차분을 수행하고 난 후, 데이터가 붉은 점선인 0을 중심으로 위아래로 불규칙하게 분포되어 있는 모습을 볼 수 있다. 이는 특정 방향으로 쏠리는 추세가 사라지고, 평균이 일정하게 유지되는 안정적인 시계열 상태로 변환되었음을 시각적으로 확인 할 수 있었다. 여기서 ADF 검정을 수행한 결과, p-value가 약 0.0507로 0.05를 미세하게 넘었지만, 시각적으로 추세가 제거되었다고 판단하였다.

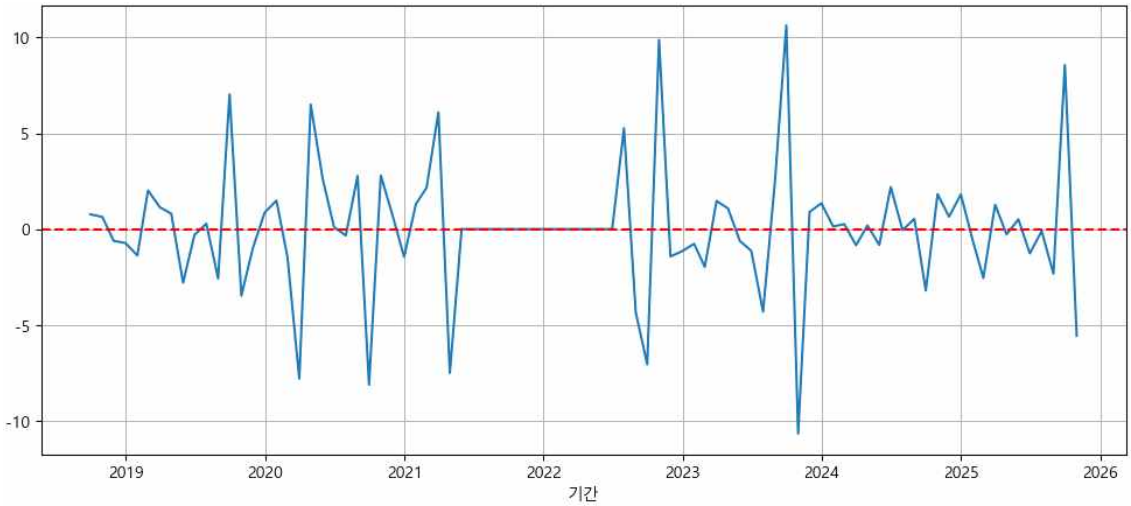


그림 2.5 계절차분(12) + 일반차분(2) 후 데이터

<그림 2.5> 는 위의 <그림 2.4>에서 p-value가 미세하게 0.05를 넘었으므로, 차분을 한 번 더 수행하면 더 안정적인 모형이 되는가를 확인하기 위해 테스트를 진행해 보았다. 1회 차분 그래프보다 y축의 변동 폭이 더 커짐을 볼 수 있다. 이는 불필요한 차분으로 인해 데이터의 분산이 오히려 증가하는 과차분 현상을 확인할 수 있었다. 따라서, 계절차분(12) + 일반차분(1) 의 데이터가 최적의 정상성 단계라고 판단했다.

2.2 ACF / PACF 분석

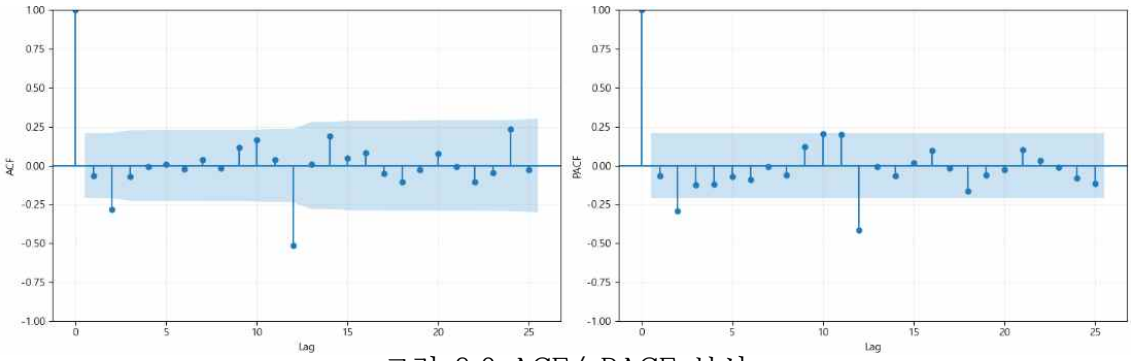


그림 2.6 ACF/ PACF 분석

모형의 차수를 정하기 위해 ACF와 PACF 분석을 수행하였다. 오른쪽 그래프가 ACF 분석 그래프이고, 왼쪽 그래프가 PACF 그래프이다. 두 그래프가 Lag 12에서 신뢰구간 밖으로 강하게 돌출되는 모습이 관찰되었다. 즉, 뚜렷한 계절성이 있다는 증거이다. 시차 1에서도 유의미한 스파이크가 관찰되는데, 이는 바로 전 달의 데이터와도 상관관계가 있음을 시사한다. 따라서, 단순 ARIMA 모형이 아닌 계절성 AR(P) 와 계절성 MA(Q) 항을 포함하는 SARIMA 모형을 최종 후보군으로 선정하고, 최적의 피라미터를 탐색하기로 하였다.

2.3 SARIMA 모형

표 2.1 SARIMA d=1 , AR(1) , MA(1)

SARIMA d = 1 AR(1) , MA(1)	AIC	BIC	Q	AR	AR.L2	AR.L3	MA	MA.L2	MA.L3	AR.S	MA.S
SARIMA (1,1,0)(1,1,1,12)	319.813	329.030	0.88	0.384	-	-	-	-	-	0.000	0.007
SARIMA (1,1,1)(1,1,1,12)	313.204	324.656	0.63	0.124	-	-	0.012	-	-	0.000	0.014
SARIMA (0,1,1)(1,1,1,12)	313.377	322.539	0.79	-	-	-	0.273	-	-	0.000	0.009
SARIMA (0,1,0)(1,1,1,12)	318.667	325.579	0.60	-	-	-	-	-	-	0.000	0.007

먼저, 모형을 정하는 기준은 q의 값이 0.05 이상이며, 비계절과 계절항의 p값이 0.05 미만인 모형을 선정하는 것을 기준으로 하였다. <표 2.1>은 1차 비계절 1차 차분과 AR, MA의 차수를 각각 1까지만 탐색한 결과이다. SARIMA(0,1,0)(1,1,1,12)를 제외한 나머지 모형은 각 항의 값이 유의하지 않았다.

표 2.2 SARIMA d=1 , AR(2) , MA(2)

SARIMA d = 1 AR(2) , MA(2)	AIC	BIC	Q	AR	AR.L2	AR.L3	MA	MA.L2	MA.L3	AR.S	MA.S
SARIMA (1,1,2)(1,1,1,12)	306.387	320.047	0.92	0.005	-	-	0.021	0.005	-	0.000	0.003
SARIMA (0,1,2)(1,1,1,12)	306.824	318.207	0.93	-	-	-	0.628	0.009	-	0.000	0.007
SARIMA (2,1,0)(1,1,1,12)	309.999	321.451	0.99	0.444	0.016	-	-	-	-	0.000	0.011
SARIMA (2,1,1)(1,1,1,12)	311.985	325.728	0.99	0.909	0.030	-	0.939	-	-	0.000	0.012
SARIMA (2,1,2)(1,1,1,12)	310.663	326.600	0.95	0.502	0.453	-	0.973	0.666	-	0.000	0.011

<표 2.2>는 1차 비계절 차분을 하고, AR, MA의 차수를 각각 2까지 탐색한 결과이다. SARIMA(1,1,2)(1,1,1,2)를 제외한 나머지 모형은 각 항의 값이 유의하지 않음을 볼 수 있었다.

표 2.3 SARIMA d=1 , AR(3) , MA(3)

SARIMA d = 1 AR(3) , MA(3)	AIC	BIC	Q	AR	AR.L2	AR.L3	MA	MA.L2	MA.L3	AR.S	MA.S
SARIMA (0,1,3)(1,1,1,12)	304.720	318.296	0.92	-	-	-	0.691	0.000	0.245	0.000	0.009
SARIMA (1,1,3)(1,1,1,12)	307.062	322.900	0.96	0.881	-	-	0.842	0.024	0.745	0.000	0.007
SARIMA (2,1,3)(1,1,1,12)	307.191	325.292	0.94	0.078	0.006	-	0.501	0.047	0.282	0.000	0.006
SARIMA (3,1,0)(1,1,1,12)	308.341	322.001	0.98	0.548	0.000	0.578	-	-	-	0.000	0.003
SARIMA (3,1,1)(1,1,1,12)	310.692	326.628	0.97	0.993	0.584	0.994	0.991	-	-	0.000	0.011
SARIMA (3,1,2)(1,1,1,12)	310.344	328.557	0.93	0.613	0.657	0.706	0.700	0.253	-	0.000	0.012
SARIMA (3,1,3)(1,1,1,12)	308.972	329.336	0.99	0.920	0.784	0.734	0.903	0.450	0.769	0.000	0.008

<표 2.3>은 비계절 1차 차분을 하고, AR , MA의 차수를 각각 3까지 탐색한 결과이다. 모든 모형의 값이 유의하지 않으므로, 3까지만 탐색하고 앞서 두 모델인 SARIMA(0,1,0)(1,1,1,12) , SARIMA(1,1,2)(1,1,1,12) 모델을 예비 모형으로 선정하였다.

<표 2.4> 예비 모델 비교표

	AIC	BIC	Q	AR	AR.L2	AR.L3	MA	MA.L2	MA.L3	AR.S	MA.S
SARIMA (0,1,0)(1,1,1,12)	318.667	325.579	0.60	-	-	-	-	-	-	0.000	0.007
SARIMA (1,1,2)(1,1,1,12)	306.387	320.047	0.92	0.005	-	-	0.021	0.005	-	0.000	0.003

기준에 적합한 두 모형이다. 두 모델을 비교할 것이므로, SARIMA(0,1,0)(1,1,1,12) 모형을 먼저 분석하여 보았다.

3. 모형진단 및 검증

3.1 모수 유의성 검정

<표 2.4>에서, SARIMA(0,1,0)(1,1,1,12)의 모형을 보면 AIC=318.667 , BIC=325.579 인 것을 확인 할 수 있다. q값이 0.6으로 0.05 이상이었으며, 계절항의 값이 0.000 , 0.007 이므로 0.05보다 작아 유의함을 볼 수 있었다.

3.2 잔차 확인

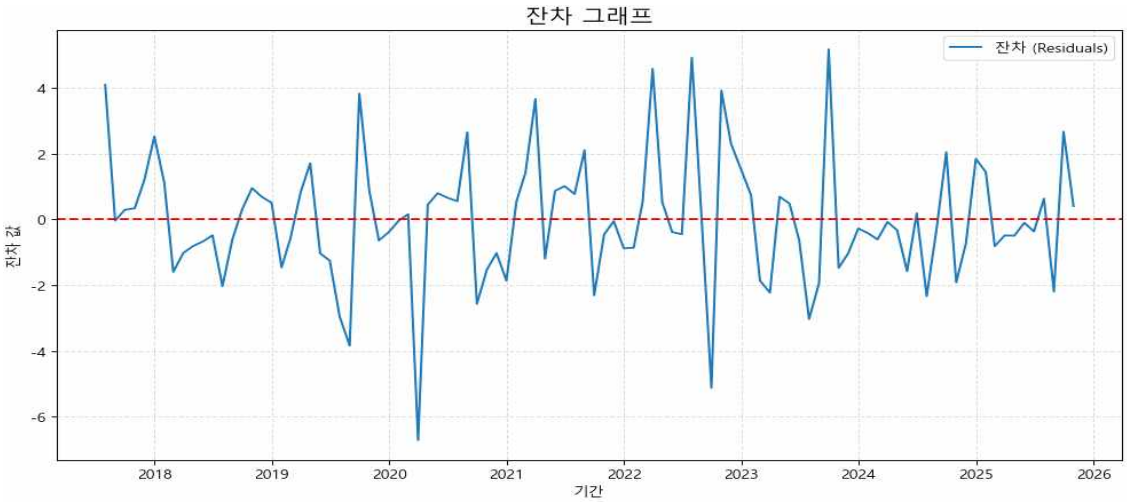


그림 3.1SARIMA(0,1,0)(1,1,1,12) 잔차 그래프

<그림 3.1>은 SARIMA(0,1,0)(1,1,1,12)의 잔차 그래프이다. 잔차가 붉은 점선인 0을 중심으로 위아래로 불규칙하게 분포해 있는 것을 시각적으로 확인 할 수 있다.

3.3 잔차의 ACF / PACF 검정

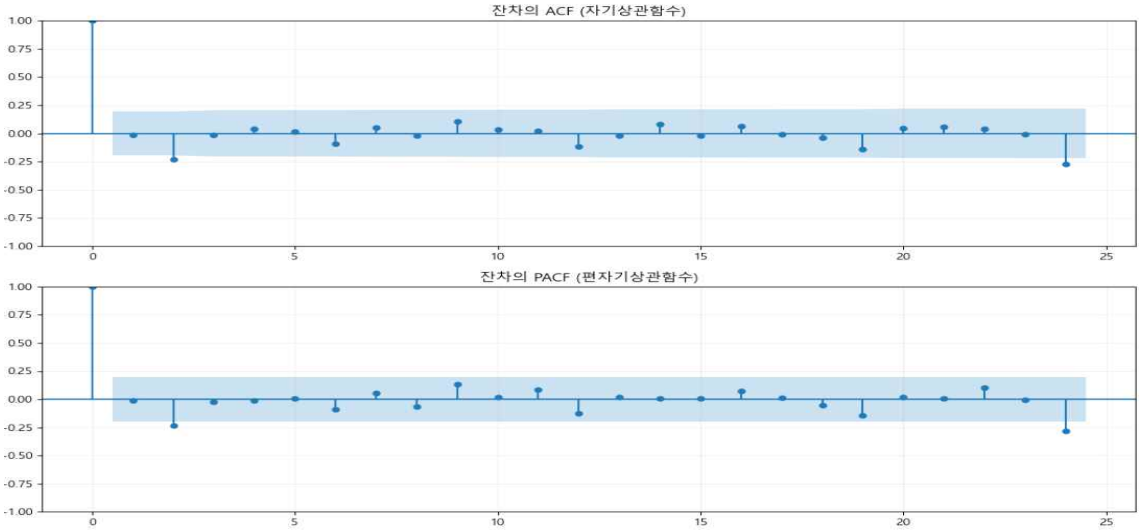


그림 3.2 잔차의 ACF / PACF 검정

<그림 3.2>의 그래프를 보면, 대부분의 막대 Lag가 파란색 음영으로 표시된 95% 신뢰구간 안쪽에 안정적으로 위치하고 있음을 확인할 수 있었다. 앞서 원본 데이터의 ACF / PACF 분석 시에는 Lag 12에서 강한 상관관계가 나타났던 것과는 달리, 현재 잔차에서는 유의미한 자기상관이나 계절적 패턴이 모두 제거되었음을 알 수 있다.

3.4 잔차의 포트맨토 검정

표 3.1 잔차의 포트맨토 검정

	lb_stat	lb_pvalue
12	10.139675	0.603708

마지막으로, <표3.1>에서 포트맨토 결과 값이 0.60으로 0.05 이상인 것을 확인하였다. 시각적 그래프 분석 뿐만 아니라 통계적인 검정 수치로도 데이터의 특성을 잘 반영하는 모형이라고 판단하였다.

3.5 모형 비교

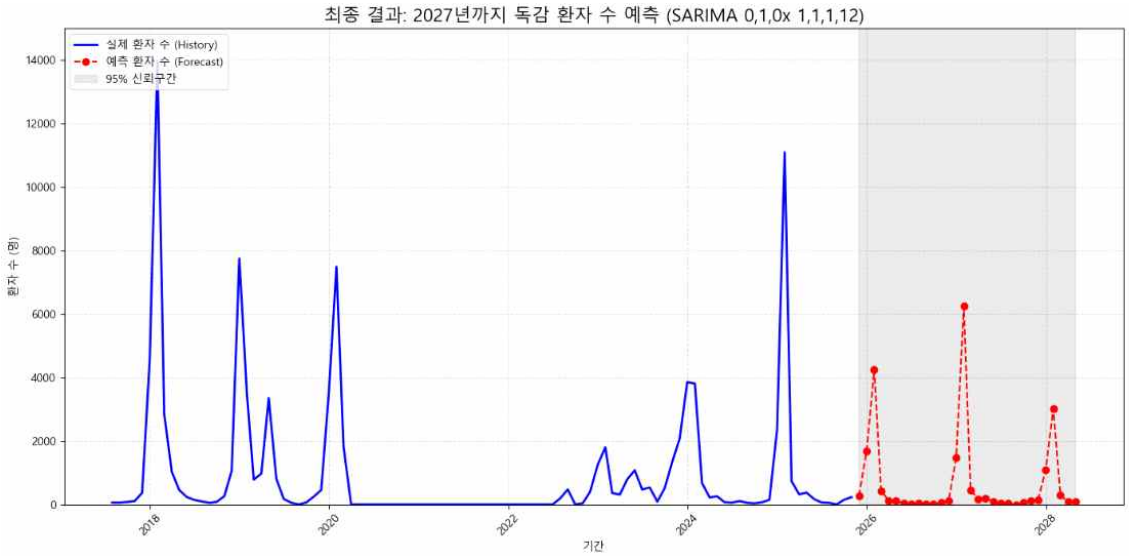


그림 3.3 SARIMA(0,1,0)(1,1,1,12)의 독감 환자 수 예측

<그림 3.3>은 SARIMA(0,1,0)(1,1,1,12)의 2027년까지의 독감 환자 수 예측 그래프이다. 파란색 실선은 과거 데이터이고, 빨간색 점선은 향후 2년 반동안의 예측 데이터이다. 로그 변환으로 이분산성을 해결하고, 계절 차분과 일반 차분을 통해 정상성을 확보한 후 모델링을 진행하였다. 그 결과, 모델은 겨울마다 환자가 치솟는 독감 특유의 계절성을 정확하게 포착하여 미래를 그렸다. 단순히 지난 달 값을 따라가려는 것이 아니라, 여름에는 0에 수렴하고, 겨울에는 약 6,000명까지 오르는 뚜렷한 '산 모양'을 스스로 만들어 내었다는 점에서 모델이 데이터의 구조를 잘 이해하였다고 판단하였다.

다음으로 SARIMA(1,1,2)(1,1,1,12) 분석을 진행하여 비교해 보았다.
다시 <표 2.4>에서, SARIMA(1,1,2)(1,1,1,12)의 모형을 보면 AIC=306.387 , BIC=320.047 인 것을 확인 할 수 있다. q값이 0.92로 0.05 이상이었으며, 비계절 항과 계절항의 값이 각각 0.000 , 0.003 이므로 0.05미만으로 통계적으로 유의함을 볼 수 있었다.

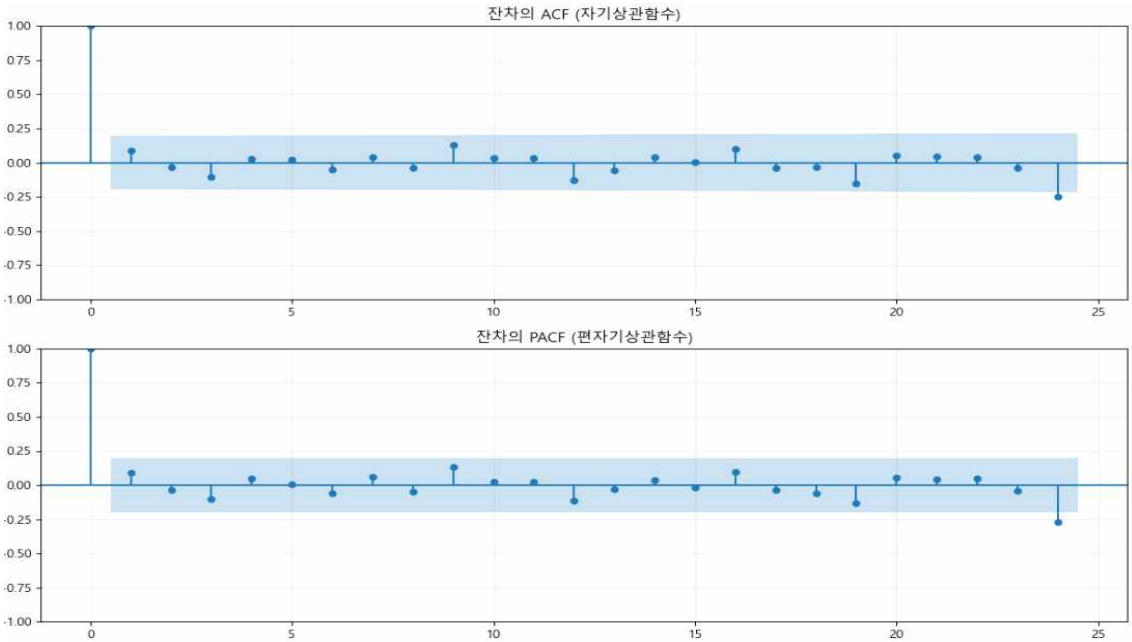


그림 3.4 잔차의 ACF / PACF 검정

<그림 3.4>는 SARIMA(1,1,2)(1,1,1,12)의 잔차 ACF / PACF 검정 결과이다. 대부분의 막대 Lag가 파란색 음영으로 표시된 95% 신뢰구간 안에 안정적으로 위치하고 있음을 확인하였다.

표 3.2 잔차의 포트맨토 검정

	lb_stat	lb_pvalue
12	6.912698	0.863328

<표3.2>는 SARIMA(1,1,2)(1,1,1,12)의 잔차 포트맨토 검정 결과이다. p값이 0.86으로 마찬가지로 0.05 이상임을 확인하였으며, SARIMA(1,1,2)(1,1,1,12) 모형 또한 시각적 그래프 분석 뿐만 아니라, 통계적 검정 수치로도 데이터의 특성을 가장 잘 반영하는 모형이라고 판단 할 수 있었다.

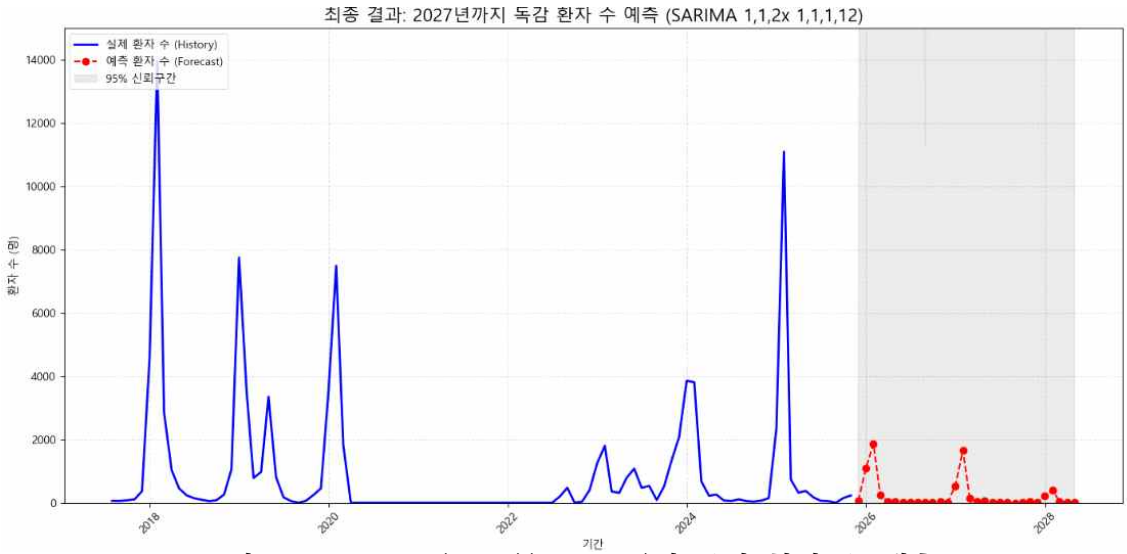


그림 3.5 SARIMA(1,1,2)(1,1,1,12)의 독감 환자 수 예측

<그림 3.5>는 SARIMA(1,1,2)(1,1,1,12)의 2027년까지의 독감 환자 수 예측 그래프이다. 이 모델은 최근의 과도한 유행을 이상치로 판단하고, 향후에는 예년 평균 수준인 2,000명대로 안정화될 것임을 보수적으로 예측했다. 이는 과대 예측을 방지하는 효과가 있다.

4. 최종모형

최종 모델 선정 단계에서 SARIMA(0,1,0)(1,1,1,12) 모형과, SARIMA(1,1,2)(1,1,1,12)모형을 두고 정밀 비교를 수행하였다.

SARIMA(1,1,2)(1,1,1,12)모형의 AIC 값이 더 낮았으나, 과거의 급증 데이터를 일시적 오차로 판단하여, 미래 유행 규모를 평균 수준으로 과도하게 낮추는 평균 회귀 경향이 강하였다. 이는 감염병 대비 관점에서 과소 예측의 위험이 있다.

반면, SARIMA(0,1,0)(1,1,1,12) 모형은 최근의 유행 추세를 가장 현실적으로 반영하였으며, 겨울철마다 확실하게 급증한다는 독감의 고유한 패턴을 가장 잘 보존하고 있었다.

따라서, 실제 방역 대책 수립에 유의미한 현실적인 유행 패턴을 예측하는 것이 더 중요하다고 판단하여 SARIMA(0,1,0)(1,1,1,12) 모형을 최종 모형으로 선정하였다.

최종 선정된 SARIMA(0,1,0)(1,1,1,12)의 수식은

$$(1-\Phi_1 B^{12})(1-B)(1-B^{12})Y_t = (1+\Theta_1 B^{12})\epsilon_t$$

이다.

표 4.1 최종 모형 대입값

	coef
ar.S.L12	-0.8252
ma.S.L12	0.4561

<표 4.1>은 AR과 MA의 값이다. 계절 AR 계수는 -0.8252로 도출되었다. 이는 12개월 전의 유행 패턴이 현재에 강한 영향을 미치되, 음수 부호를 통해 과도한 변동을 억제하고 균형을 맞추려는 성질을 보여준다. 또한, 계절 MA 계수는 0.4561로, 과거의 예측 오차를 약 45% 반영하여 다음 예측을 보정한다. 결국 이 수식은, 작년의 유행 패턴을 기본 골격으로 가져오며, AR과 MA 항을 통해 오차를 보정하여 미래를 예측하는 메커니즘이라고 요약된다. 이 AR과 MA 값을 대입하면,

$$Y_t = (Y_{t-1} + Y_{t-12} - Y_{t-13}) - 0.8252 \cdot Z_{t-12} + 0.4561 \cdot \epsilon_{t-12}$$

이와 같이 수식으로 나타낼 수 있다.

5. 결론 및 향후 과제

본 연구는 여러 SARIMA 모형을 비교한 결과, 최종적으로 가장 적합한 예측 모형은 SARIMA(0,1,0)(1,1,1,12)로 선정하였다. 일반 차분과 계절 차분을 모두 수행하며, 잔차의 ACF / PACF 및 포트맨토 검정을 통해 잔차가 자기상관이 없는 백색 잡음인 것을 확인하여 모형의 적합성을 통계적으로 검증하였다.

이 과정에서, 최종 수식은 전년도의 그래프 형태가 미래 예측에 반영되는 구조적 특성을 가지게 되었다. 예측 결과, 2026년과 2027년의 독감 환자 수는 향후 겨울철에도 집중적으로 늘어나는 계절성을 유지하는 것으로 나타났다. 반면, 겨울철이 아닌 기간에는 독감이 거의 유행하지 않거나, 환자 수가 낮게 유지되는 시기가 존재함을 확인할 수 있었다.

하지만 이러한 특성은 오히려 감염병 예방 관점에서는 강점이 될 수 있다. 독감은 매년 비슷한 시기에 반복되는 계절성이 매우 강력한 질병이기 때문이다.

향후 과제로는 독감이 본격적으로 유행하기 시작하는 11월 이전에 대응 전략을 마련하는 것이 필요하다. 특히, 독감 예방 주사 캠페인을 실시하거나 예방 주사 필요성 교육, 포스터나 온라인 마케팅을 통해 예방 체계를 강화할 수 있을 것이다.

또한, 병원 및 보건소의 경우 예측 결과를 바탕으로 독감 백신의 재고를 사전에 충분히 확보하고 효율적으로 재고를 관리하는 전략을 수립할 수 있을 것으로 기대된다. 본 연구의 시계열 예측 결과는 향후 보건 자원 배분 및 감염병 대응 계획 수립에 유효한 지표가 될 수 있을 것이다.

부 록

< Python Code >

```
#1.2 데이터 설명
import pandas as pd
df = pd.read_excel("독감.xlsx", sheet_name="Sheet1")
print(df.head())
df.rename(columns={'Unnamed: 0': 'season'}, inplace=True)
print(df.head())
df.info()
from influenza_helpers import attach_dates, weekly_to_monthly
df['season'] = df['season'].str.replace('~', '-')
df = attach_dates(df, season_col='season', week_col='week')
print(df.head(10))
monthly = weekly_to_monthly(df, value_col='value', how='sum')
print(monthly.head(12))
```

```
#그림1.2 월 별 인플루엔자 환자수 추이
import matplotlib.pyplot as plt
from matplotlib import font_manager, rc
import platform
if platform.system() == 'Windows':
font_path = "C:/Windows/Fonts/malgun.ttf"
font_name =
font_manager.FontProperties(fname=font_path).get_name()
rc('font', family=font_name)
else:
rc('font', family='AppleGothic')
plt.rcParams['axes.unicode_minus'] = False
import matplotlib.pyplot as plt
plt.figure(figsize=(12, 5))
plt.plot(monthly['month'], monthly['value'],
marker='o', linewidth=2)
plt.title('월별 인플루엔자 환자수 추이 (2017-2025)',
fontsize=14, fontweight='bold')
plt.xlabel('월', fontsize=12)
plt.ylabel('환자 수', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

```
#그림2.1 원본 데이터
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
plt.figure(figsize=(12, 5))
plt.plot(monthly_final['month'], target_data, color='green',
label='Raw Data (No Log)')
plt.title("원본 데이터 ADF 검정", fontsize=15)
plt.xlabel('기간')
plt.ylabel('환자 수 (명)')
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
plt.show()
print("=== 원본 데이터(로그 X, 차분 X) ADF 검정 결과
===")
result = adfuller(target_data.dropna())
print(f"ADF Statistic: {result[0]:.4f}")
print(f"p-value: {result[1]:.5f}")
print("-" * 40)
if result[1] < 0.05:
```

```
#그림2.2 로그 변환 후 월별 독감 환자수
from statsmodels.tsa.stattools import adfuller
print("

target_data = monthly_final['log_value'].dropna()
result = adfuller(target_data)
print(f"ADF Statistic: {result[0]:.4f}")
print(f"p-value: {result[1]:.5f}")
print("-" * 30)
if result[1] < 0.05:
```

```
#그림2.3 계절 차분(12) 후 데이터
from statsmodels.tsa.stattools import adfuller
monthly_final['log_diff_12']
= monthly_final['log_value'].diff(12)
plt.figure(figsize=(12, 5))
plt.plot(monthly_final['month'], monthly_final['log_diff_12'])
plt.title
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel("기간")
plt.ylabel("계절 차분값")
plt.grid(True)
plt.xticks(rotation=45)
plt.show()
target_data = monthly_final['log_diff_12'].dropna()
result = adfuller(target_data)
print("=== 계절 차분 후 ADF 검정 결과 ===")
print(f"ADF Statistic: {result[0]:.4f}")
print(f"p-value: {result[1]:.5f}")
if result[1] < 0.05:
```

```
#그림2.4 계절차분(12) + 일반차분(1) 후 데이터
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
monthly_final['log_diff_12_1'] = monthly_final['log_diff_12'].diff(1)
plt.figure(figsize=(12, 5))
plt.plot(monthly_final['month'], monthly_final['log_diff_12_1'])
plt.title("계절차분(12) + 일반차분(1) 적용 후 ADF")
plt.axhline(y=0, color='r', linestyle='--') # 0점 기준선
plt.xlabel("기간")
plt.ylabel("변동값")
plt.grid(True)
plt.xticks(rotation=45)
plt.show()
target_data = monthly_final['log_diff_12_1'].dropna()
result = adfuller(target_data)
print("=== 최종 차분(계절+일반) 후 ADF 검정 결과 ===")
print(f"ADF Statistic: {result[0]:.4f}")
print(f"p-value: {result[1]:.5f}")
if result[1] < 0.05:
```

```
#그림2.5 계절차분(12) + 일반차분(2) 후 데이터
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
monthly_final['log_diff_12_2']
= monthly_final['log_diff_12_1'].diff(1)
plt.figure(figsize=(12, 5))
plt.plot(monthly_final['month'],
monthly_final['log_diff_12_2'])
plt.title("일반 차분 2회 적용 후 과차분 확인")
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel("기간")
plt.grid(True)
plt.show()
target_data_d2 = monthly_final['log_diff_12_2'].dropna()
result_d2 = adfuller(target_data_d2)
print(f"=== 일반 차분 2회 (d=2) ADF 결과 ===")
print(f"ADF Statistic: {result_d2[0]:.4f}")
print(f"p-value: {result_d2[1]:.5f}")
std_d1 = monthly_final['log_diff_12_1'].std()
std_d2 = monthly_final['log_diff_12_2'].std()
print("\n=== 과차분 판단 (표준편차 비교) ===")
print(f"d=1 일 때 표준편차: {std_d1:.4f}")
print(f"d=2 일 때 표준편차: {std_d2:.4f}")
if std_d2 > std_d1:
```

```
#그림2.6 ACF / PACF 검정
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf,
plot_pacf
target_data = monthly_final['log_diff_12_1'].dropna()
fig, ax = plt.subplots(1, 2, figsize=(15, 5))
plot_acf(target_data, lags=25, ax=ax[0])
ax[0].set_title("ACF - 차분된 데이터 (MA 차수 확인)")
ax[0].set_xlabel("Lag")
ax[0].set_ylabel("ACF")
ax[0].grid(True, alpha=0.3)
plot_pacf(target_data, lags=25, ax=ax[1], method='ywm')
ax[1].set_title("PACF - 차분된 데이터 (AR 차수 확인)")
ax[1].set_xlabel("Lag")
ax[1].set_ylabel("PACF")
ax[1].grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
#표2.1 SARIMA d=1, AR(1) , MA(1)
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.stats.diagnostic import acorr_ljungbox
print("=== 수정된 모델: SARIMA (0, 1, 1) x (1, 1, 1, 12) 검증 ===")
model_v2 = SARIMAX(monthly_final['log_value'],
order=(0, 1, 1),
seasonal_order=(1, 1, 1, 12),
enforce_stationarity=False,
enforce_invertibility=False)
results_v2 = model_v2.fit(disp=False)
print(results_v2.summary())
print("\n" + "="*50)
print(">>> [재검증] 유의하지 않은(p>0.05) 항이 있는지 체크합니다:")
print("="*50)
for term, p_val in results_v2.pvalues.items():
if term != 'sigma2':
if p_val > 0.05:
print(f"⚠여전히 문제: '{term}' 항의 p-value가 {p_val:.4f}입니다.")
else:
print(f"☑통과: '{term}' 항의 p-value는 {p_val:.4f}입니다.
(아주 좋음!)")
print("\n=== 잔차 포트맨토 검정 (Ljung-Box) ===")
lb_result = acorr_ljungbox(results_v2.resid, lags=[12],
return_df=True)
print(lb_result)
```

```
#표2.2 SARIMA d=1, AR(2) , MA(2)
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.stats.diagnostic import acorr_ljungbox
print("=== 수정된 모델: SARIMA (1, 1, 2) x (1, 1, 1, 12) 검증 ===")
model_v2 = SARIMAX(monthly_final['log_value'],
order=(1, 1, 2),
seasonal_order=(1, 1, 1, 12),
enforce_stationarity=False,
enforce_invertibility=False)
results_v2 = model_v2.fit(disp=False)
print(results_v2.summary())
print("\n" + "="*50)
print(">>> [재검증] 유의하지 않은(p>0.05) 항이 있는지 체크합니다:")
print("="*50)
for term, p_val in results_v2.pvalues.items():
if term != 'sigma2':
if p_val > 0.05:
```

```
#표2.3 SARIMA d=1, AR(3) , MA(3)
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.graphics.tsaplots import plot_acf,
plot_pacf
from statsmodels.stats.diagnostic import acorr_ljungbox
print("=== 수정된 모델: SARIMA (0, 1, 3) x (1, 1, 1, 12) 검
증 ===")
model_v2 = SARIMAX(monthly_final['log_value'],
order=(0, 1, 3),
seasonal_order=(1, 1, 1, 12),
enforce_stationarity=False,
enforce_invertibility=False)
results_v2 = model_v2.fit(disp=False)
print(results_v2.summary())
print("\n" + "="*50)
print(">>> [재검증] 유의하지 않은(p>0.05) 항이 있는지 체크합
니다:")
print("="*50)
for term, p_val in results_v2.pvalues.items():
if term != 'sigma2':
if p_val > 0.05:
```

```
#표2.4 예비 모델 비교표
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.graphics.tsaplots import plot_acf,
plot_pacf
from statsmodels.stats.diagnostic import acorr_ljungbox
print("=== 수정된 모델: SARIMA (0, 1, 1) x (1, 1, 1, 12) 검
증 ===")
model_v2 = SARIMAX(monthly_final['log_value'],
order=(0, 1, 1),
seasonal_order=(1, 1, 1, 12),
enforce_stationarity=False,
enforce_invertibility=False)
results_v2 = model_v2.fit(disp=False)
print(results_v2.summary())
print("\n" + "="*50)
print(">>> [재검증] 유의하지 않은(p>0.05) 항이 있는지 체크합
니다:")
print("="*50)
for term, p_val in results_v2.pvalues.items():
if term != 'sigma2':
if p_val > 0.05:
```

```
#그림3.1 SARIMA(0,1,0)(1,1,1,12) 잔차 그래프
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX
import datetime
file_name = "최종 엑셀 가공본.xlsx"
monthly_bonghap = pd.read_excel(file_name)
def make_date_from_week(row):
start_year = int(str(row['year']).split('~')[0])
week = int(row['week'])
if week >= 30: year = start_year
else: year = start_year + 1
try: return pd.to_datetime(f'{year}-{week}-1',
format='%Y-%W-%w')
except: return pd.NaT
monthly_bonghap['date']
= monthly_bonghap.apply(make_date_from_week, axis=1)
monthly_bonghap
= monthly_bonghap.dropna(subset=['date'])
monthly_final
= monthly_bonghap.set_index('date')
.resample('ME')['value'].sum().reset_index()
monthly_final.columns
= ['month', 'value']
monthly_final['log_value'] = np.log1p(monthly_final['value'])
print("모델 학습 중...")
model = SARIMAX(monthly_final['log_value'],
order=(0, 1, 0),
seasonal_order=(1, 1, 1, 12),
enforce_stationarity=False,
enforce_invertibility=False)
final_result = model.fit(disp=False)
residuals = final_result.resid
plt.figure(figsize=(12, 6))
plt.plot(monthly_final['month'], residuals, label='잔차 (Residuals)')
plt.axhline(y=0, color='r', linestyle='--', linewidth=1.5)
plt.title('잔차 그래프', fontsize=15)
plt.xlabel('기간')
plt.ylabel('잔차 값')
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
plt.show()
```

```
#그림3.2 잔차의 ACF / PACF 검정
#표3.1 잔차의 포트맨토 검정
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.graphics.tsaplots import plot_acf,
plot_pacf
from statsmodels.stats.diagnostic import acorr_ljungbox
print("=== 📓 최종 모형 건강검진 시작 ===")
model_final = SARIMAX(monthly_final['log_value'],
order=(0, 1, 0),
seasonal_order=(1, 1, 1, 12),
enforce_stationarity=False,
enforce_invertibility=False)
results_final = model_final.fit(disp=False)
print("\n[1] 모수 유의성 검정 결과")
print(results_final.summary())
print("\n>>> P-value 확인:")
all_pass = True
for term, p_val in results_final.pvalues.items():
if term != 'sigma2': # 오차분산 제외
if p_val < 0.05:
if all_pass
residuals = results_final.resid
plt.figure(figsize=(12, 8))
plt.subplot(2, 1, 1)
plot_acf(residuals, lags=24, ax=plt.gca(), title="잔차의 ACF
(자기상관함수)")
plt.grid(True, alpha=0.3)
plt.subplot(2, 1, 2)
plot_pacf(residuals, lags=24, ax=plt.gca(), title="잔차의
PACF (편자기상관함수)")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
print("\n[3] 포트맨토 검정 (Ljung-Box Test)")
# 계절성이 12이므로 lags=[12]로 검정하는 것이 일반적입니다.
lb_test = acorr_ljungbox(residuals, lags=[12],
return_df=True)
print(lb_test)
lb_pvalue = lb_test['lb_pvalue'].values[0]
print("-" * 40)
if lb_pvalue > 0.05:
```

```
#그림3.3 SARIMA(0,1,0)(1,1,1,12)의 독감 환자 수 예측
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from statsmodels.tsa.statespace.sarimax import SARIMAX
print("=== SARIMA(0,1,0)(1,1,1,12) 모델 학습 및 예측 ===")
model = SARIMAX(monthly_final['log_value'],
order=(0, 1, 0),
seasonal_order=(1, 1, 1, 12),
enforce_stationarity=False,
enforce_invertibility=False)
final_result = model.fit(disp=False)
forecast = final_result.get_forecast(steps=30)
pred_log = forecast.predicted_mean
conf_int_log = forecast.conf_int()
pred_value = np.expm1(pred_log)
lower_limits = np.expm1(conf_int_log.iloc[:, 0])
upper_limits = np.expm1(conf_int_log.iloc[:, 1])
pred_value[pred_value < 0] = 0
lower_limits[lower_limits < 0] = 0
upper_limits[upper_limits < 0] = 0
plt.figure(figsize=(14, 7))
last_date = monthly_final['month'].iloc[-1]
prediction_dates
= pd.date_range(start=last_date, periods=31, freq='ME')[1:]
plt.plot(monthly_final['month'],
np.expm1(monthly_final['log_value']),
label='실제 환자 수 (History)', color='blue', linewidth=2)
plt.plot(prediction_dates, pred_value,
label='예측 환자 수 (Forecast)',
color='red', linestyle='--', marker='o')
plt.fill_between(prediction_dates,
lower_limits, upper_limits,
color='gray', alpha=0.2, label='95% 신뢰구간')
plt.ylim(0, 15000)
plt.title("최종 결과: 2027년까지 독감 환자 수 예측
(SARIMA 0,1,0x 1,1,1,12)", fontsize=16)
plt.xlabel("기간")
plt.ylabel("환자 수 (명)")
plt.legend(loc='upper left')
plt.grid(True, linestyle='--', alpha=0.5)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
print("\n=== 📋 2026년~2027년 겨울철 예상 환자 수 ===")
print(pred_value.tail(15))
```

```
#그림3.4 잔차의 ACF / PACF 검정
#표3.2 잔차의 포트맨토 검정
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.graphics.tsaplots import plot_acf,
plot_pacf
from statsmodels.stats.diagnostic import acorr_ljungbox
print("=== 🔬 최종 모형 건강검진 시작 ===")
model_final = SARIMAX(monthly_final['log_value'],
order=(1, 1, 2),
seasonal_order=(1, 1, 1, 12),
enforce_stationarity=False,
enforce_invertibility=False)
results_final = model_final.fit(disp=False)
print("\n[1] 모수 유의성 검정 결과")
print(results_final.summary())
print("\n>>> P-value 확인:")
all_pass = True
for term, p_val in results_final.pvalues.items():
if term != 'sigma2': # 오차분산 제외
if p_val < 0.05:
if all_pass
residuals = results_final.resid
plt.figure(figsize=(12, 8))
plt.subplot(2, 1, 1)
plot_acf(residuals, lags=24, ax=plt.gca(), title="잔차의 ACF
(자기상관함수)")
plt.grid(True, alpha=0.3)
plt.subplot(2, 1, 2)
plot_pacf(residuals, lags=24, ax=plt.gca(), title="잔차의
PACF (편자기상관함수)")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
print("\n[3] 포트맨토 검정 (Ljung-Box Test)")
# 계절성이 12이므로 lags=[12]로 검정하는 것이 일반적입니다.
lb_test = acorr_ljungbox(residuals, lags=[12],
return_df=True)
print(lb_test)
lb_pvalue = lb_test['lb_pvalue'].values[0]
print("-" * 40)
if lb_pvalue > 0.05:
```

```
#그림3.5 SARIMA(1,1,2)(1,1,1,12)의 독감 환자 수 예측
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from statsmodels.tsa.statespace.sarimax import SARIMAX
print("=== SARIMA(1,1,2)(1,1,1,12) 모델 학습 및 예측 ===")
model = SARIMAX(monthly_final['log_value'],
order=(1, 1, 2),
seasonal_order=(1, 1, 1, 12),
enforce_stationarity=False,
enforce_invertibility=False)
final_result = model.fit(disp=False)
forecast = final_result.get_forecast(steps=30)
pred_log = forecast.predicted_mean
conf_int_log = forecast.conf_int()
pred_value = np.expm1(pred_log)
lower_limits = np.expm1(conf_int_log.iloc[:, 0])
upper_limits = np.expm1(conf_int_log.iloc[:, 1])
pred_value[pred_value < 0] = 0
lower_limits[lower_limits < 0] = 0
upper_limits[upper_limits < 0] = 0
plt.figure(figsize=(14, 7))
last_date = monthly_final['month'].iloc[-1]
prediction_dates
= pd.date_range(start=last_date, periods=31, freq='ME')[1:]
plt.plot(monthly_final['month'],
np.expm1(monthly_final['log_value']),
label='실제 환자 수 (History)', color='blue', linewidth=2)
plt.plot(prediction_dates, pred_value,
label='예측 환자 수 (Forecast)',
color='red', linestyle='--', marker='o')
plt.fill_between(prediction_dates,
lower_limits, upper_limits,
color='gray', alpha=0.2, label='95% 신뢰구간')
plt.ylim(0, 15000)
plt.title("최종 결과: 2027년까지 독감 환자 수 예측
(SARIMA 1,1,2x 1,1,1,12)", fontsize=16)
plt.xlabel("기간")
plt.ylabel("환자 수 (명)")
plt.legend(loc='upper left')
plt.grid(True, linestyle='--', alpha=0.5)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
print("\n=== 📋 2026년~2027년 겨울철 예상 환자 수 ===")
print(pred_value.tail(15))
```